UCAM

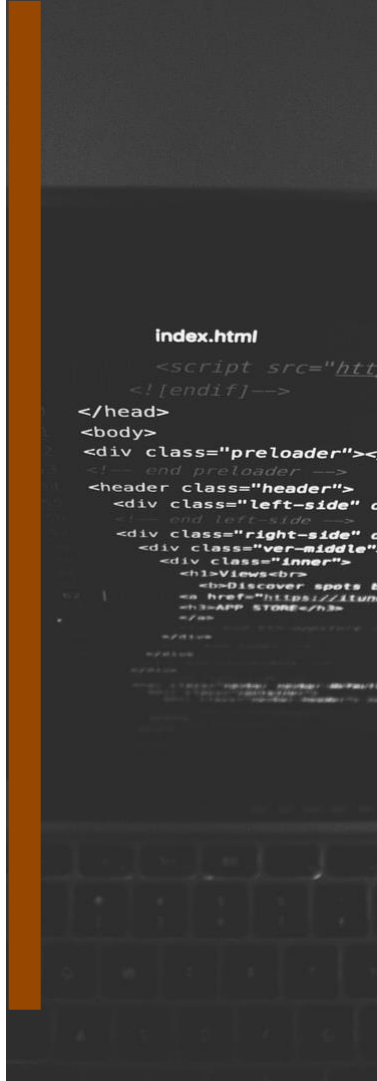
Tema 2

Conceptos básicos en C

Fundamentos de Programación I

Juan Bonastre Egea / Angélica Guzmán Ponce

Grado en Ingeniería Informática



ÍNDICE

CONTENIDOS

1. Elementos de un programa en C
2. Directivas
3. Funciones
4. Sentencias
5. Cadenas de texto
6. Comentarios

ÍNDICE

CONTENIDOS

7. Variables de programa
8. Entrada de datos
9. Constantes
10. Identificadores y palabras reservadas

1. Elementos de un programa en C

Imagen 1
Portátil semiabierto



Nota. Adaptado de *Glowing Laptop*, por Joshua Reddekopp, 2020, Unsplash

1. Elementos de un programa en C

Nuestro primer programa en C

```
#include <stdio.h>
int main(void)
{
    printf("A programar se aprende programando.\n");
    return 0;
}
```

1. Elementos de un programa en C

Forma general de un programa C

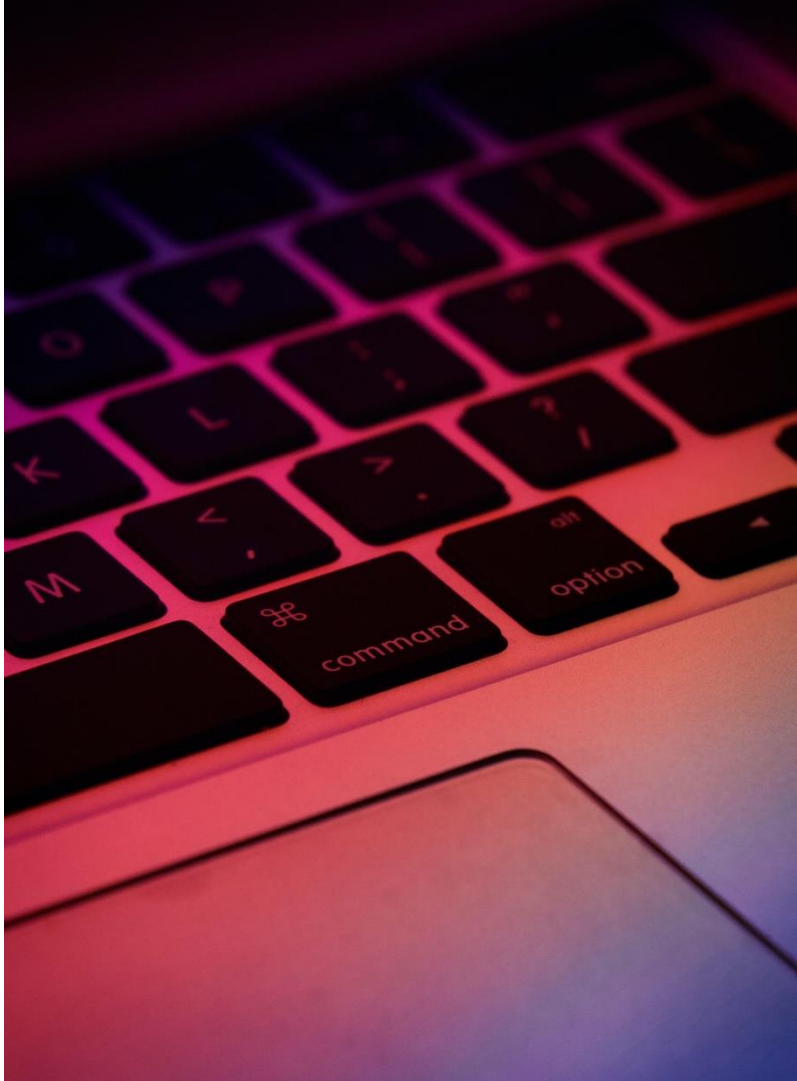
- ✓ La forma de un programa en C es:

```
<< directivas >>
int main(void)
{
    << sentencias >>
}
```

- ✓ Incluso el programa más sencillo debe tener siempre:
  - **Directivas.** Comandos que modifican el programa antes de compilar.
  - **Funciones.** Bloques de código ejecutable (como la función main).
  - **Sentencias.** Instrucciones a ejecutar cuando se lance el programa.

2. Directivas

Imagen 2
Primer plano de un teclado de un portátil con luces de colores



Nota. Adaptado de *Black and silver laptop computer*, por Manthan Gajjar, 2020, Unsplash

2. Directivas

- ✓ Antes de compilar un programa, tiene que ser editado por el **preprocesador**.
- ✓ Los comandos que el preprocesador es capaz de entender se denominan **directivas**.
- ✓ Las directivas siempre empiezan con el carácter **#**
- ✓ Sobre las directivas:
  - ✓ Una directiva por línea.
  - ✓ Nunca terminan ni con **;** ni con otro carácter especial.

2. Directivas

- ✓ Por ahora solo nos interesa la directiva:

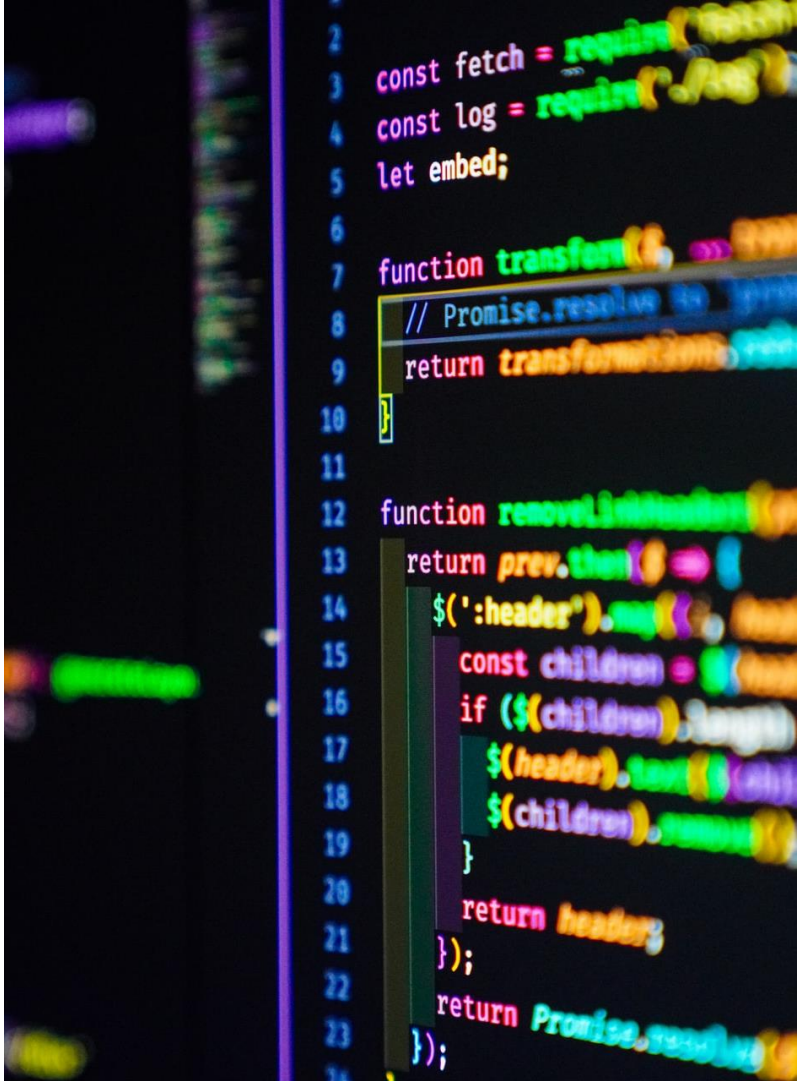
```
#include  <stdio.h>
```

  - Esta directiva indica al preprocesador que incluya la información contenida en el fichero **stdio.h** en nuestro programa.
  - **stdio.h** contiene información sobre la librería estándar de E/S de C.

- ✓ El lenguaje C no incorpora ninguna función para mostrar información por pantalla o para leer datos de teclado. Si queremos realizar E/S debemos utilizar las funciones de la librería estándar de C, por lo que debemos incluir el fichero cabecera (**header**) "`stdio.h`".

3. Funciones

Imagen 3
Lenguaje de programación



Nota. Adaptado de *Javascript program*, por Joan Gamell, 2020, Unsplash

3. Funciones
Functions

- ✓ Las funciones son los **bloques de construcción** que se usan en C para escribir programas.
  - Un programa en C no es más que un conjunto de funciones.
  - Otros lenguajes las denominan procedimientos o subrutinas.
- ✓ Las podemos clasificar en:
  - Funciones escritas por el programador.
  - Funciones proporcionadas como parte de la implementación de C.
    - Denominadas funciones de librería (**library functions**).

3. Funciones
Functions

- ✓ El término función proviene del concepto matemático de función.
  - Una función es un conjunto de reglas para calcular un valor a partir de otros valores que se proporcionan como argumentos.
  - Ejemplos:

```
f(x) = x + 1
```

```
g(y, z) = y² * z²
```

- ✓ C utiliza el concepto de función de una forma más **flexible**, menos rigurosa que las Matemáticas.

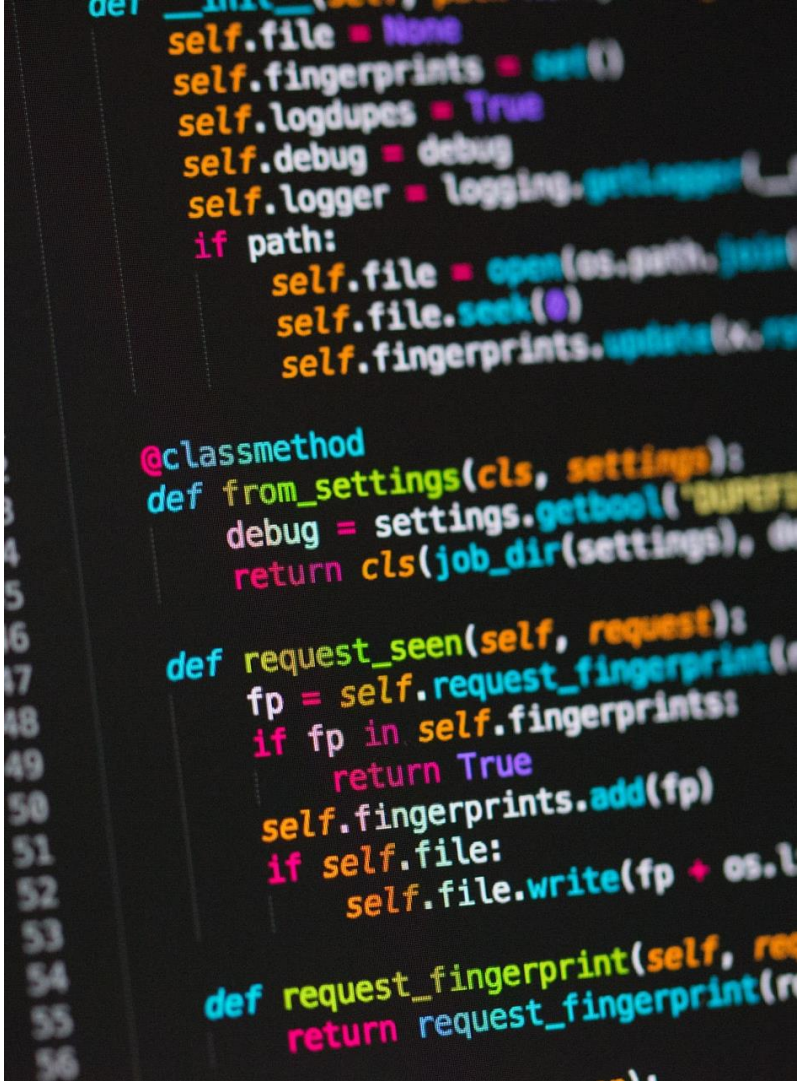
3. Funciones
Functions

- ✓ **`main`** debe devolver un valor, concretamente un entero.
- ✓ **`main`** puede no aceptar parámetros (Aunque esto NO es del todo correcto).
- ✓ La sentencia **`return`** tiene dos funcionalidades:
  - Hace que la función **main** termine (y por lo tanto el programa).
  - Devolviendo un cero, indica que la ejecución fue correcta.

```
#include <stdio.h>
int main(void)
{
    printf("A programar se aprende programando.\n");
    return 0;
}
```

4. Sentencias

Imagen 4
Lenguaje de programación



Nota. Adaptado de *Free Code Image*, por Chris Ried, 2018, Unsplash

4. Sentencias

- ✓ Una sentencia es un comando a ser ejecutado cuando lanzamos nuestro programa.
  - Todas las sentencias deben **terminar con el carácter ;**
  - El ; indica al compilador dónde termina la sentencia.
- ✓ El ejemplo anterior tan solo tenía dos sentencias:

```
printf("A programar se aprende programando.\n");
```

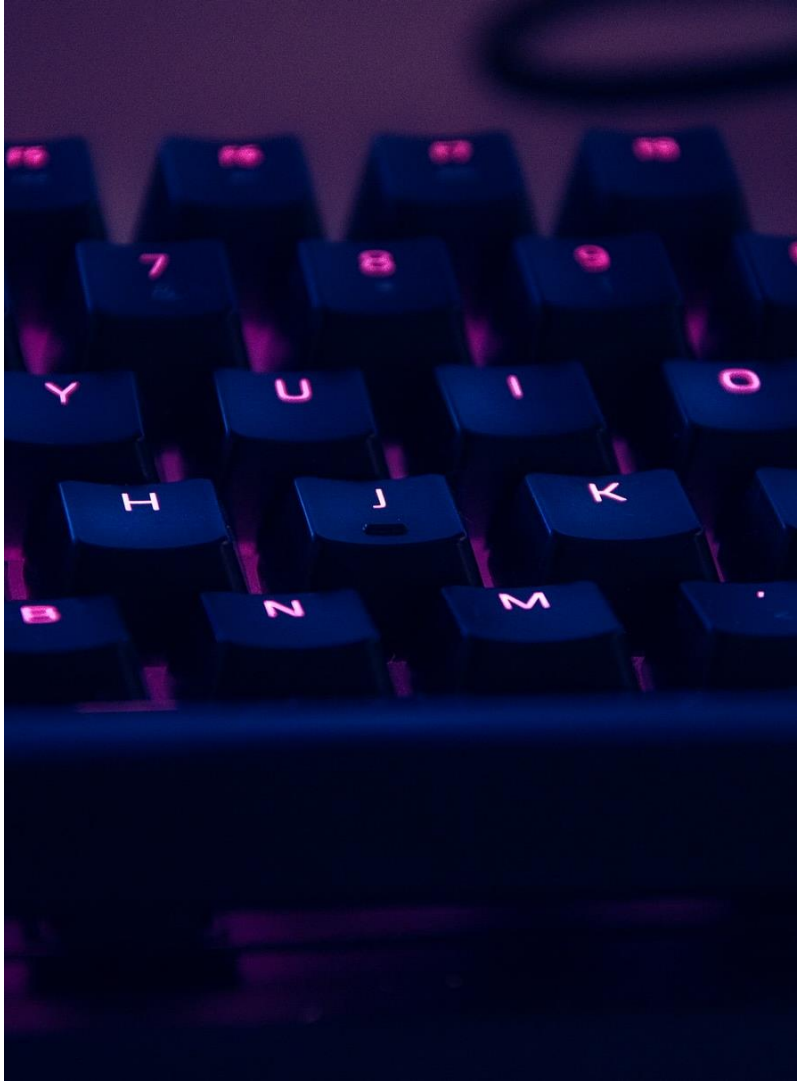
  - Llamada a la función printf que muestra un mensaje en pantalla.

```
return 0;
```

  - Terminación del programa.
- ✓ La ejecución de sentencias se realiza de forma secuencial.

5. Cadenas de texto

Imagen 5
Teclado de ordenador



Nota. Adaptado de *Focus photography of computer keyboard*, por Anas Alshanti, 2016, Unsplash

5. Cadenas de texto

Mostrando cadenas de texto

- ✓ **printf** es una función muy potente que es utilizada para mostrar cadenas de caracteres.
  - Una cadena de caracteres (string) es una serie de caracteres cerrados entre comillas dobles:

```
"Esto es un ejemplo de cadena."
""     (cadena vacía)
"Otro ejemplo de cadena.\n"
```

5. Cadenas de texto

Mostrando cadenas de texto

- `printf` no imprime retornos de carro a no ser que se lo indiquemos explícitamente:

  Esta llamada a `printf`:

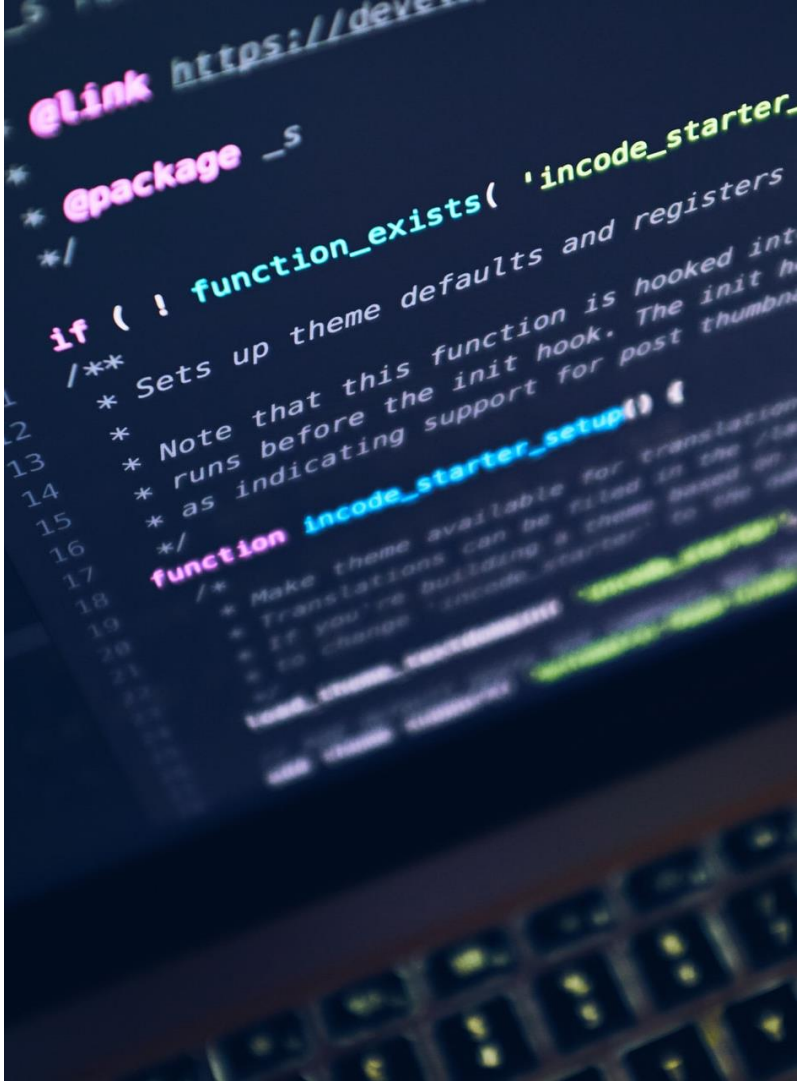
  ```
  printf("Ser o no ser: esa es la cuestión.\n");
  ```

  … produce la misma salida que estas dos llamadas:

  ```
  printf("Ser o no ser: ");
  printf("esa es la cuestión.\n");
  ```

6. Comentarios

Imagen 6
Código en una pantalla



Nota. Adaptado de *Code on a laptop screen*, por Luca Bravo, 2017, Unsplash

6. Comentarios

- ✓ Hasta ahora nuestro programa carece de algo importante: **documentación**.
  - Es importante indicar en todos nuestros programas, al menos:
    - Nombre del programa.
    - Fecha en la que fue escrito.
    - Autor.
    - Qué hace el programa.
  - En C toda la información adicional que queramos incluir se realiza a través de comentarios.
  - Los comentarios son ignorados por el compilador.

6. Comentarios

- ✓ Los comentarios se encierran entre los caracteres **/*** y ***/**

```
/* Esto es un comentario */
/*
 * File:    hola.c
 * Author:  Juan Bonastre / Angélica Guzmán
 * Purpose: Mostrar un mensaje de texto en pantalla.
 * Created on 10 de septiembre de 2026, 9:30
 */
```

6. Comentarios

También podemos insertar comentarios en nuestro código:

```
int main(void)
```

```
/* Empezamos el programa */
```

- Los comentarios ayudan a mantener el código.
- Debe ser una ayuda para todo aquel que vaya a leer nuestro código, incluido nosotros mismos.

Con el estándar C99 se incluyó el estilo de comentarios de C++:

```
// Esto es un comentario al estilo de C++.
```

Los caracteres **//** hacen que el compilador ignore todo lo que haya desde los caracteres **//** **hasta el final de la línea**.

6. Comentarios

- ✓ Si utilizamos los comentarios /* … */, es fácil cometer el error de no cerrar el comentario y que el compilador ignore parte de nuestro programa:

```
printf("Ojo");     /* Este comentario no se cierra ...
printf("con");
printf("no cerrar");
printf("los");  /* ... hasta que cerramos este otro. */
printf("comentarios!!\n");
```

- ✓ ¿Qué muestra el anterior código?

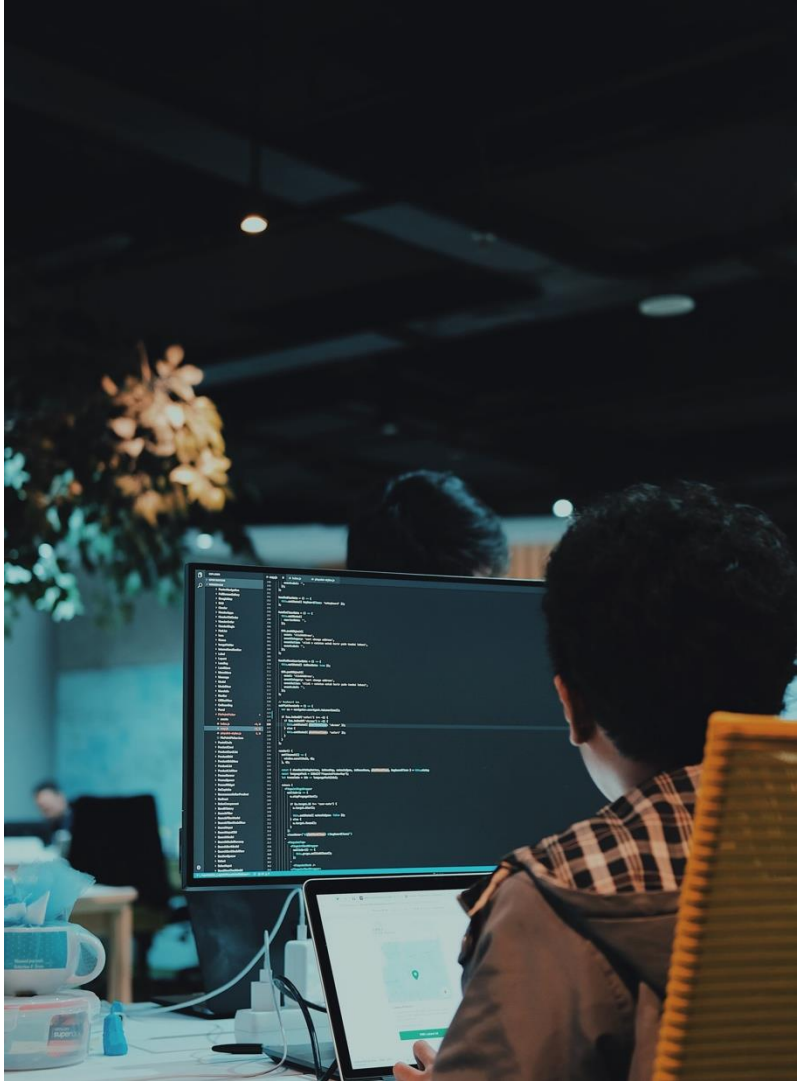
6. Comentarios

Nuestro primer programa en C

```
/*
 * File:    t2-ej2.c
 * Author:  Juan Bonastre / Angélica Guzmán
 * Purpose: Mostrar un mensaje de texto en pantalla.
 */
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    printf("Hola mundo!!\n");
    return 0;
}
```

7. Variables de programa

Imagen 7
Hombre programando



Nota. Adaptado de *A man who writes code*, por Arif Riyanto, 2018, Unsplash

7. Variables de programa

- ✓ Hasta ahora nuestros programas han sido muy simples...

  ...pero muchos programas necesitan hacer cálculos antes de mostrar información por pantalla,

- ✓ Por lo que se necesita un **lugar temporal donde almacenar dichos resultados intermedios**.

- ✓ En C, y en otros muchos lenguajes, esos lugares de almacenamiento temporal se denominan **variables**.

7. Variables de programa

Tipos de variables

- ✓ Cada variable debe tener un tipo (**type**).
  - ✓ El tipo de una variable determina qué clase de datos puede almacenar.
- ✓ La elección del tipo es crítica ya que determina:
  - ✓ **Cómo se almacena** la variable.
  - ✓ **Qué tipo de operaciones** podemos realizar.
- ✓ El tipo de una variable numérica determina, además:
  - ✓ El número **más grande** y el número **más pequeño** que podemos almacenar.
  - ✓ Si podemos almacenar números **decimales** (reales).

7. Variables de programa

Tipos de variables

- ✓ Por ahora vamos a limitar el número de tipos que vamos a manejar a:
  - **int** (integer, entero)
    - Puede almacenar **números enteros**, **tales como 0, 2345 ó -2.**
  - **float** (floating point, coma flotante)
    - Puede almacenar **números decimales** (reales) como el -234.8287, así como números enteros (como el 1.0).
    - Sin embargo, la aritmética en coma flotante es mucho más lenta que la aritmética entera.
    - Además, se pueden producir errores de redondeo.

7. Variables de programa
Declaración de variables

- ✓ Las variables deben ser declaradas antes de poder ser utilizadas.
  - C es un lenguaje tipado y el compilador necesita saber el tipo de las variables.
- ✓ Para **declarar una variable**:
  - Especificamos el tipo.
  - Le asignamos un nombre.
- ✓ Ejemplos:

```
int altura;
int ancho;
float superficie;
```

7. Variables de programa
Declaración de variables

- ✓ Podemos combinar varias declaraciones en la misma línea:

```
int x, y, z;
```

- ✓ La forma de un programa en C sería entonces:

```
<< directivas >>
int main(void)
{
    << declaración variables programa >>
    << sentencias >>
}
```

7. Variables de programa

Asignación de valores a variables

- ✓ Una variable puede tomar un valor a través de una asignación.

  Las sentencias...

```
base = 18;
altura = 12;
```

  ...asignan valores a las variables base y altura. Los números 18 y 12 se denominan 'constantes'.

  - Antes de asignar un valor a una variable debemos declararla:

```
int base;
base = 18;
```

7. Variables de programa

Asignación de valores a variables

- ✓ Una constante asignada a una variable tipo float debe contener un punto decimal.
  - Si superficie es una variable tipo float podemos asignarle un valor así:

```
superficie = 32.67;
```

Normalmente:

- Asignaremos valores de tipo *int* a variables de tipo *int*.
- Asignaremos valores de tipo *float* a variables de tipo *float*.
- Es posible **mezclar tipos** *(int y float)* pero no siempre se obtendrán los resultados esperados.

7. Variables de programa

Asignación de valores a variables

- ✓ Una vez que hemos asignado valores a las variables, podemos utilizarlas para calcular otros valores:

```
largo = 34;
ancho = 23;
alto = 9;
     volumen = largo * ancho * alto;
   /* Volumen vale ahora 7038 */
```

Tema 6

- En C, '*****' representa la multiplicación. Otros: + - / %
- La sentencia multiplica las variables largo, ancho y alto, almacenando el resultado en volumen.
- La parte derecha de una asignación puede ser cualquier expresión que contenga constantes, variables y operadores.

7. Variables de programa

Mostrando el valor de una variable

- ✓ Para mostrar el valor de una variable se utiliza también la función **`printf`**.

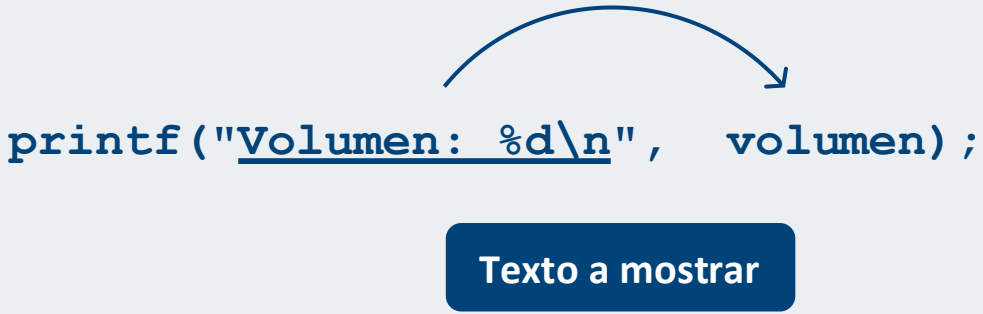
```
Para escribir el mensaje:
                    " Volumen: v "
... utilizaríamos la siguiente llamada a printf:
          printf("Volumen: %d\n", volumen);
```

Tema 3

- **`%d`** es un hueco que se utiliza para indicar que se va a mostrar un valor entero, dicho valor se 'cubrirá' con el valor de la variable volumen.
- **`\n`** indica que se muestre además un salto de línea.
- **`%d`** solo funciona con enteros, para mostrar números con decimales utilizaremos **`%f`**.

7. Variables de programa

Mostrando el valor de una variable



7. Variables de programa

Mostrando el valor de una variable

- ✓ ...pero printf también es capaz de mostrar el resultado de cualquier expresión numérica:

  Para escribir el mensaje:

  `Volumen: v`

  ...podemos utilizar también la llamada:

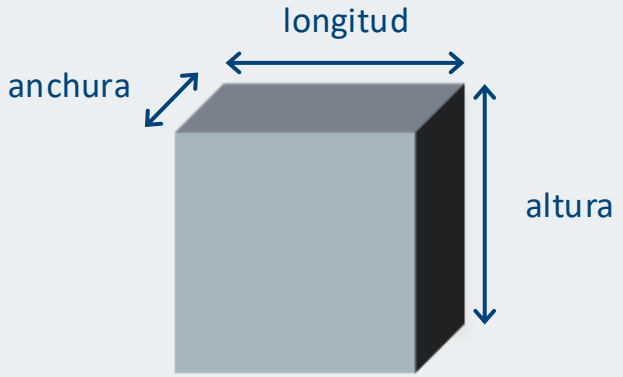


```
printf("Volumen:%d\n",largo*ancho*alto);
```

7. Variables de programa

Ejemplo: volumen de una caja

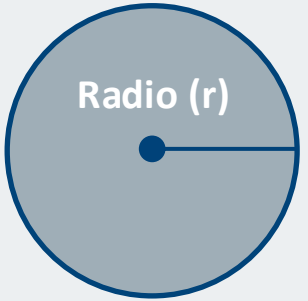
```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    int largo;
    int ancho;
    int alto;
    int volumen;
    largo = 34;
    ancho = 23;
    alto = 9;
    volumen = largo * ancho * alto;
    printf("Dimensiones %d x %d x %d\n", largo, ancho, alto);
    printf("El volumen de la caja es: %d\n", volumen);
    return 0;
}
```



7. Variables de programa

Ejemplo: longitud de una circunferencia

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    int radio;
    float pi = 3.14159265359;
    float longitud;
    float superficie;
    radio = 18;
    longitud = 2 * pi * radio;
    superficie = pi * radio * radio;
    printf("Radio: %d Longitud circunferencia: %f\n", radio, longitud);
    printf("Radio: %d Superficie circulo: %f\n", radio, superficie);
    return 0;
}
```



7. Variables de programa
Inicialización de variables

- ✓ A algunas variables se les asigna el valor cero de forma automática cuando el programa empieza a ejecutarse, pero otras muchas variables no.
- ✓ Una variable que no tiene un valor por defecto y que no se le ha asignado un valor se dice que está **no inicializada**.
  - Mostrar el valor de una variable no inicializada puede devolver cualquier valor extraño `(-1208042064, 54298...)` o incluso, depende del compilador, que el programa falle o se cuelgue.
  - Solo las variables globales y estáticas se inicializan a 0 automáticamente; las variables locales NO: contienen un valor indeterminado (basura) hasta que se les asigna algo.

7. Variables de programa
Inicialización de variables

- ✓ Siempre podemos asignar un valor a una variable a través de una asignación, pero también se puede hacer en la misma declaración de la variable:

```
int altura = 23;
```

  - En la jerga C se dice que 23 es un inicializador.
- ✓ Varias variables se pueden inicializar durante la misma declaración:

```
int altura = 80, anchura = 10, peso = 32;
```

  - Cada variable necesita tener su propio inicializador

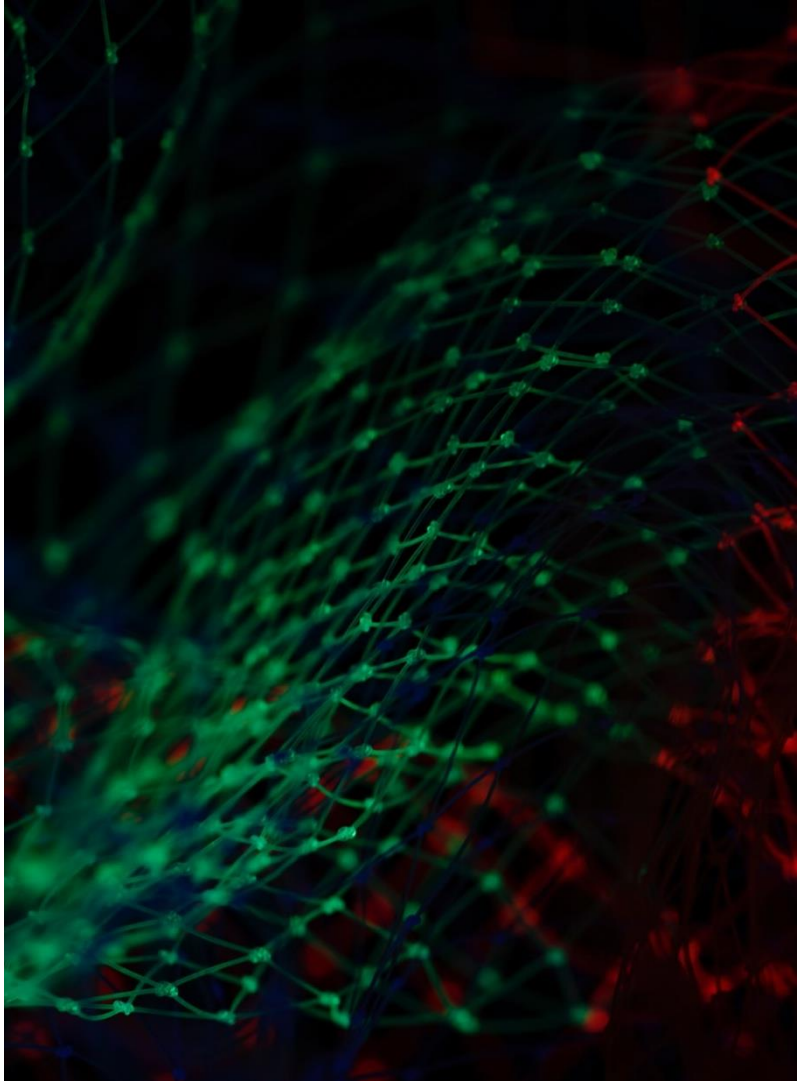
7. Variables de programa

Ejemplo: variables no inicializadas

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    int a;    // El valor de a es indeterminado
    printf("a = %d\n", a);
    return 0;
}
```

8. Entrada de datos

Imagen 8
Luces rojas, verdes y azules



Nota. Adaptado de *Green and red light*, por Pietro Jeng, 2017, Unsplash

8. Entrada de datos

Entrada de datos

- ✓ El programa que calculaba el volumen de la caja no era especialmente útil, ya que siempre calculaba el volumen de la misma caja.
- ✓ Para mejorar nuestro programa, necesitamos que el usuario sea capaz de introducir las dimensiones.
- ✓ De eso se encarga la función **`scanf`**.
  - **`scanf`** es a la entrada de datos lo que **`printf`** lo es a la salida.
  - La '`f`' final de **`scanf`** y **`printf`** significa formateada (*formatted*), ambas funciones requieren del uso de una cadena de formato para especificar la apariencia de los datos de entrada/salida.

8. Entrada de datos

Entrada de datos

- ✓ Para leer un número entero y almacenarlo en la variable i, se utiliza la función scanf de la siguiente manera:

```
scanf("%d", &i);
```

  - La cadena '**%d**' indica a `scanf` que **se va a leer un número entero**.
  - **i** es la variable en la que se va a almacenar el valor leído.
  - El carácter **&** se explicará más adelante. Normalmente lo llevan cuando se utiliza scanf.

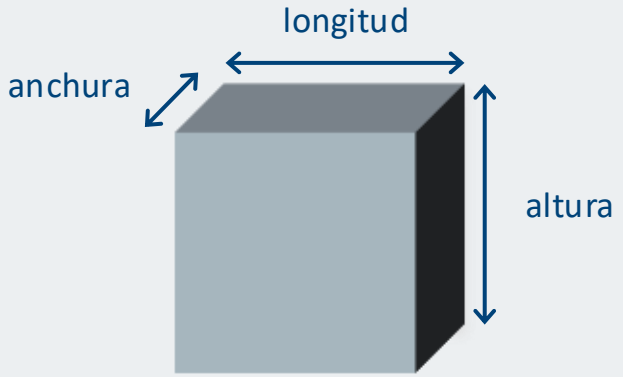
- ✓ De forma similar, se puede **leer un número real** con:

```
scanf("%f",&x);
```

8. Entrada de datos

Ejemplo: volumen de una caja v2

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    int largo;
    int ancho;
    int alto;
    int volumen;
    printf("Introduzca la longitud: ");
    scanf("%d", &largo);
    printf("Introduzca la anchura: ");
    scanf("%d", &ancho);
    printf("Introduzca la altura: ");
    scanf("%d", &alto);
    volumen = largo * ancho * alto;
    printf("Dimensiones %d x %d x %d\n", largo, ancho, alto);
    printf("El volumen de la caja es: %d\n", volumen);
    return 0;
}
```



8. Entrada de datos

Ejecución del programa

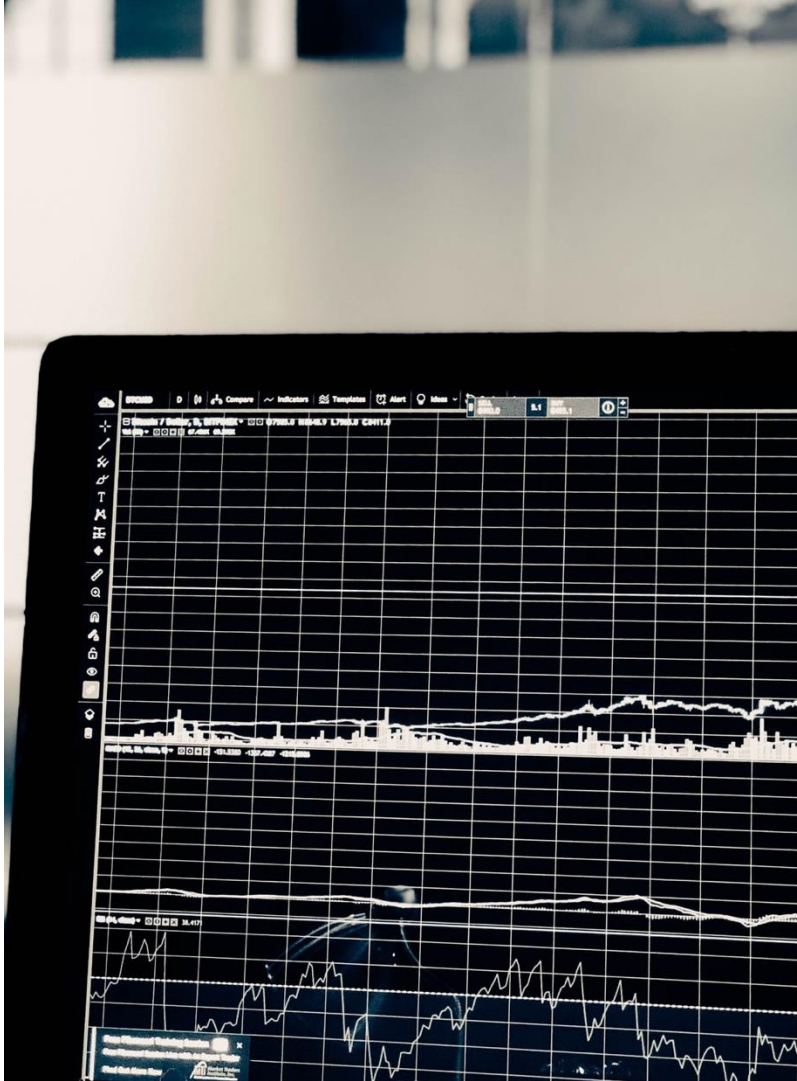
```
Introduzca la longitud: 13
Introduzca la anchura: 87
Introduzca la altura: 45
Dimensiones 13 x 87 x 45
El volumen de la caja es: 50895
```

...pero el programa tiene un fallo:

¿Qué pasa si la entrada que se teclea **no son números**?

9. Constantes

Imagen 9
Estadísticas de bitcoin



Nota. Adaptado de *Bitcoin stats*, por Chris Liverani, 2018, Unsplash

9. Constantes

- ✓ Cuando un programa contiene **constantes**, se pueden definir a través de un nombre.
  - Si la constante aparece en el programa tal cual, no puede quedar claro su significado.
  - Si la constante cambia, tan solo habrá que modificar su valor en un solo lugar.
- ✓ La declaración de constantes se realiza a través de una definición de macros, de esta forma:

```
#define CONSTANTE valor
```

  - Como la directiva **`#include`**, observar que **NO acaban con un ;**
  - Normalmente se especifican en mayúsculas.
- ✓ Se puede definir una constante como PI así:

```
#define PI 3.14159265359
```

9. Constantes

Ejemplo: conversión euro - pesetas

```
#include <stdio.h>
#include <stdlib.h>
#define PESETAS_1EURO 166.386
int main(void)
{
    float euros;
    float pesetas;
    printf("Introduzca la cantidad en euros: ");
    scanf("%f", &euros);
    pesetas = euros * PESETAS_1EURO;
    printf("%f euros = %f pesetas\n", euros, pesetas);
    return 0;
}
```

9. Constantes

Ejemplo: conversión de temperaturas

```
#include <stdio.h>
#include <stdlib.h>
#define PUNTO_CONGELACION 32.0
#define FACTOR_ESCALA (5.0 / 9.0)
int main(void)
{
    float fahrenheit, celsius;
    printf("Introduzca la temperatura en grados Fahrenheit: ");
    scanf("%f", &fahrenheit);
    celsius = (fahrenheit - PUNTO_CONGELACION) * FACTOR_ESCALA;
    printf("%f °F = %f °C\n", fahrenheit, celsius);
    return 0;
}
```

9. Constantes

- ✓ La expresión:

```
celsius = (fahrenheit - PUNTO_CONGELACION) * FACTOR_ESCALA;
```

- ✓ ...es vista por el compilador como:

```
celsius = (fahrenheit - 32.0) * (5.0 / 9.0);
```

  - Si una macro contiene operadores, debe definirse entre ( )
  - Si se fijara el factor de escala como (**5/9**) el compilador generaría el valor 0, ya que C trunca el resultado de una división entera (**5/9 vale 0, no 0.5555**).

- ✓ En la sentencia...

```
printf("%f °F = %f °C\n", fahrenheit, celsius);
```

  - **%.2f** en lugar de %f mostraría las temperaturas con solo dos decimales.

10. Identificadores y palabras reservadas

Imagen 10
Código de programación en tonos violetas



Nota. Adaptado de *Flat screen computer monitor*, por Markus Spiske, 2019, Unsplash

10. Identificadores



- ✓ Al escribir programas es imprescindible seleccionar nombres que representen funciones, variables, constantes, etiquetas o cualquier otra entidad.
- ✓ A estos nombres se les denomina **identificadores**.
- ✓ En C, **un identificador es una secuencia de letras, números y guiones bajos** (_).

10. Identificadores

Hay ciertas restricciones al elegir un identificador:

- Debe ser una **secuencia de letras y dígitos**.
- **Longitud máxima de 31 caracteres**, aunque algunos compiladores aceptan más.
- El **primer carácter** debe ser **una letra**, el símbolo _ también se considera una letra.
- Se **distingue entre mayúsculas y minúsculas**.
  - Los identificadores cuenta, Cuenta, CuEnTa y CUENTA son distintos.
- Un identificador no puede coincidir con una **palabra reservada** del lenguaje.
- Tampoco puede coincidir con el nombre de ninguna **función de la biblioteca de C**.

10. Identificadores

Ejemplos de identificadores

- ✓ Identificadores legales:

`x` `y23` `_342` `suma_5` `Suma46` `area` `VOID`

`_temperatura` `temp` `tEmP` `TABLA` `caracter` `__p`

`__564Pe` `temporal12776` `Numero_repeticiones`

- ✓ Identificadores no permitidos:

`34` `23x` `mi-nombre` `numero+2` `int` `while`

`8temp` `área` `año` `n°_año` `main` `void`

10. Identificadores

Ejemplos de identificadores

- Dígitos (0…9) y letras (A…Z, a…z, _)
- Primer carácter una letra
- Máxima longitud: 31 caracteres
- Se distingue entre mayúsculas y minúsculas
- No palabra reservada ni nombre función biblioteca C

10. Identificadores

Guía para la elección de identificadores

- ✓ Los identificadores deben **reflejar el uso** que se les está dando.
  - Ejemplos: `iva`, `importe_total`, `suma`, `apellidos`, `direccion` ….
  - ¿Para qué se está utilizando la variable `h32`?
- ✓ Si el identificador **está compuesto por varias palabras**, se utilizan dos convenciones:

1. Todos los identificadores en minúsculas (salvo las macros), las palabras se separan con el carácter _

Ej.: `base_imponible`, `suma`, `importe_total`

2. Todas las palabras juntas, cada una empezando en mayúsculas excepto la primera (estilo C++/Java).

Ej.: `baseImponible`, `suma`, `importeTotal`

10. Identificadores
Palabras reservadas

- Las palabras reservadas del lenguaje tienen un significado especial para el compilador y **NO PUEDEN ser utilizadas como identificadores**.
- Las palabras reservadas en C son:

```
auto        for             sizeof
break       goto            static
case        if              struct
char        inline (C99)    switch
const       int             typedef
continue    long            union
default     register        unsigned
do          restrict (C99)  void
double      return          volatile
else        short           while
enum        signed          _Bool (C99)
extern                      _Complex (C99)
float                       _Imaginary (C99)
```

10. Identificadores
Palabras reservadas

- ✓ Las palabras reservadas se escriben tal como aparecen en la tabla anterior.
- ✓ Cada compilador de C puede tener, además, ciertas palabras reservadas, por lo que se tendrá que evitar el uso de dichas palabras.



Práctica en el aula

Caza de errores: este programa tiene 6 errores. Encuéntralos y corrígelos

```
#include <stdio.h
int main(void)
{
    int edad
    printf("Cuantos anios tienes? );
    scanf("%d", edad);
    /* Mostramos el resultado
    printf("Tienes %d anios\n" edad);
    return 0;
}
```

Práctica en el aula

A programar se aprende programando

1. Programa con cabecera de comentarios (autor, fecha, propósito) que muestre tu nombre en pantalla.
2. Declara **`int edad`** y **`float altura`**, asígnales valores y muéstralos con **`printf`**.
3. Lee con **`scanf`** la base y la altura de un rectángulo y muestra su área.
4. Define **`#define IVA 0.21`** y calcula el precio final de un producto leído por teclado.

REFERENCIAS

BIBLIOGRÁFICAS

1. Joyanes Aguilar L. Fundamentos de programación, algoritmos, estructura de datos y objetos, 5ª edición, McGraw-Hill Interamericana de España S.L.; 2020. ISBN: 9781456279448
2. Prieto A, Lloris A y Torres JC. Introducción a la Informática. 3ªed. McGraw-Hill; 2002.

Angélica Guzmán Ponce

aguzmán@ucam.edu

UCAM Universidad Católica de Murcia



© UCAM